



Assessment 2 – Process Map Report
Subject Code: TEC305
Subject Name: Algorithms and Data Structures
Assessment Title: Process Map
Tutor: Manoj Kovalam
Student Name: Manish Ranabhat
Student ID: 1863872
Word Count: 1093, including in-text citation

Contents

Introduction 1

Process Table 2

Programming Data Structures 6

Implementing Abstract Data Types (ADTs) 13

Estimating Algorithmic Complexity..... 17

Conclusion 18

References 19

Introduction

In today’s digital world, the success of modern applications—from search engines and social media to financial systems and healthcare software—depends heavily on how efficiently they handle data. Consider a navigation app that calculates the fastest route in real time. Behind this simple task lies complex data being stored, accessed, and processed using appropriate data structures and algorithms. Without a well-designed approach to managing data, such systems would fail to perform under pressure (Shaffer 2013).

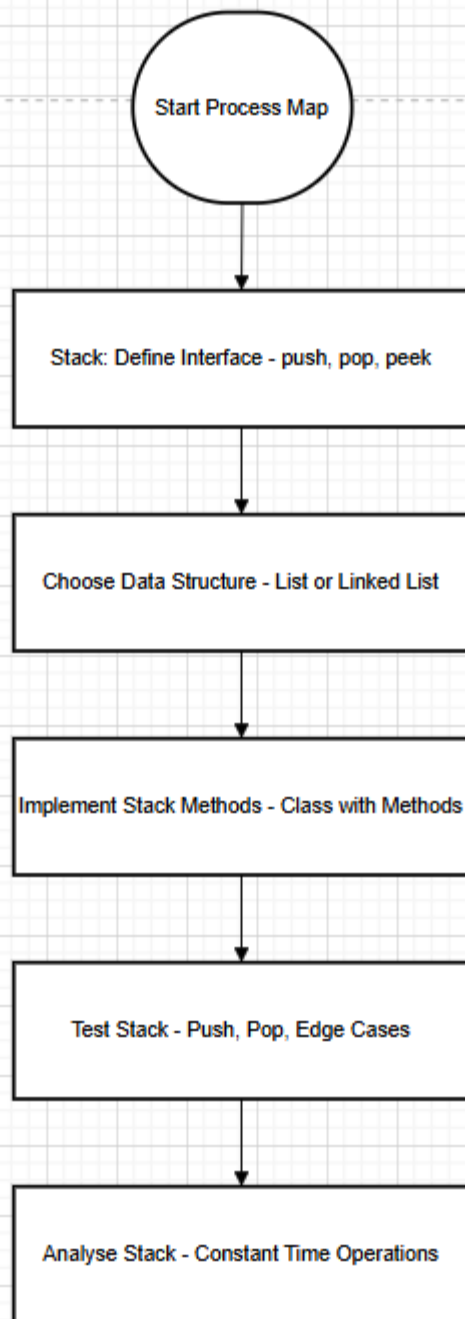
This report explores two essential data structures—Stack (an Abstract Data Type) and Binary Search Tree (a Complex Data Structure)—to demonstrate how they are constructed,

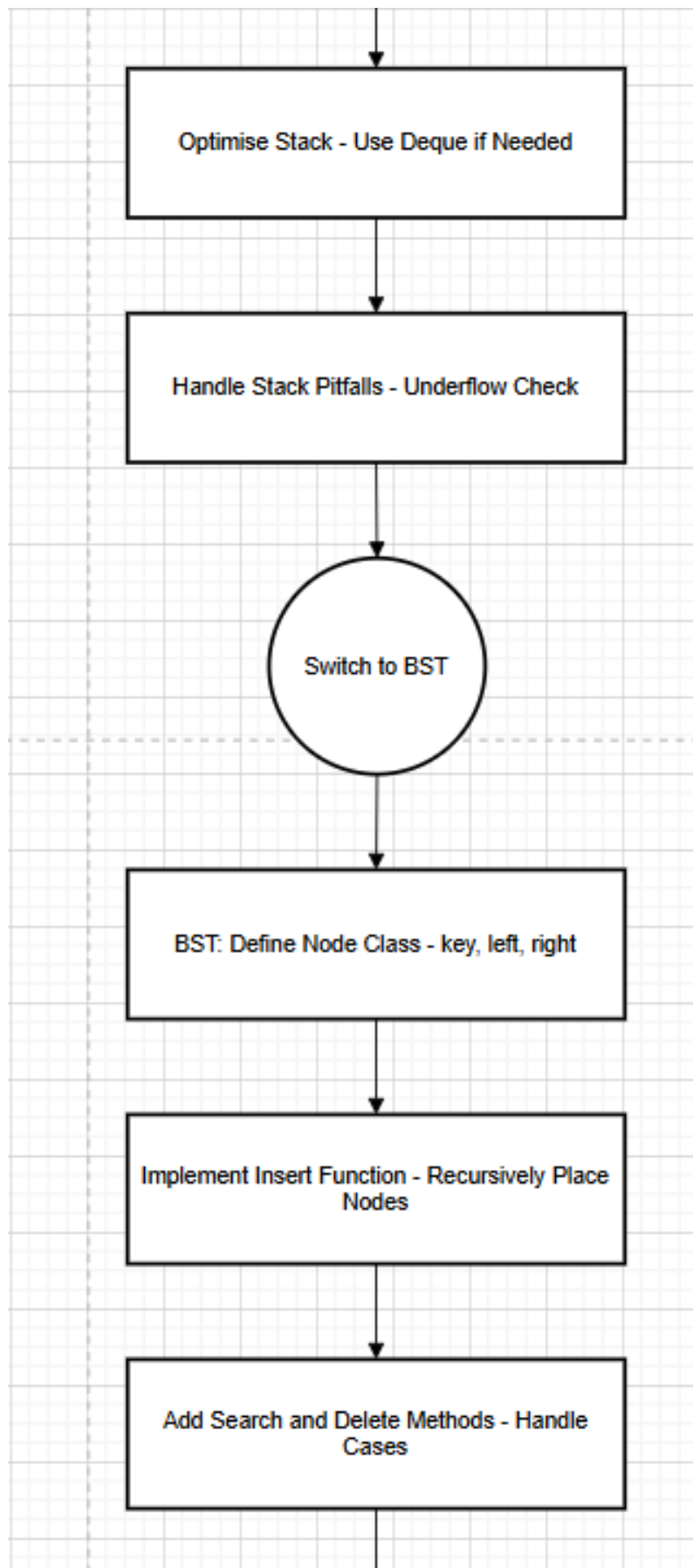
implemented, and optimized in programming workflows. This report outlines the step-by-step construction of both structures, including code implementations, testing procedures, and algorithmic complexity analysis. Additionally, a visual process map is provided to illustrate the logical flow from design to optimization.

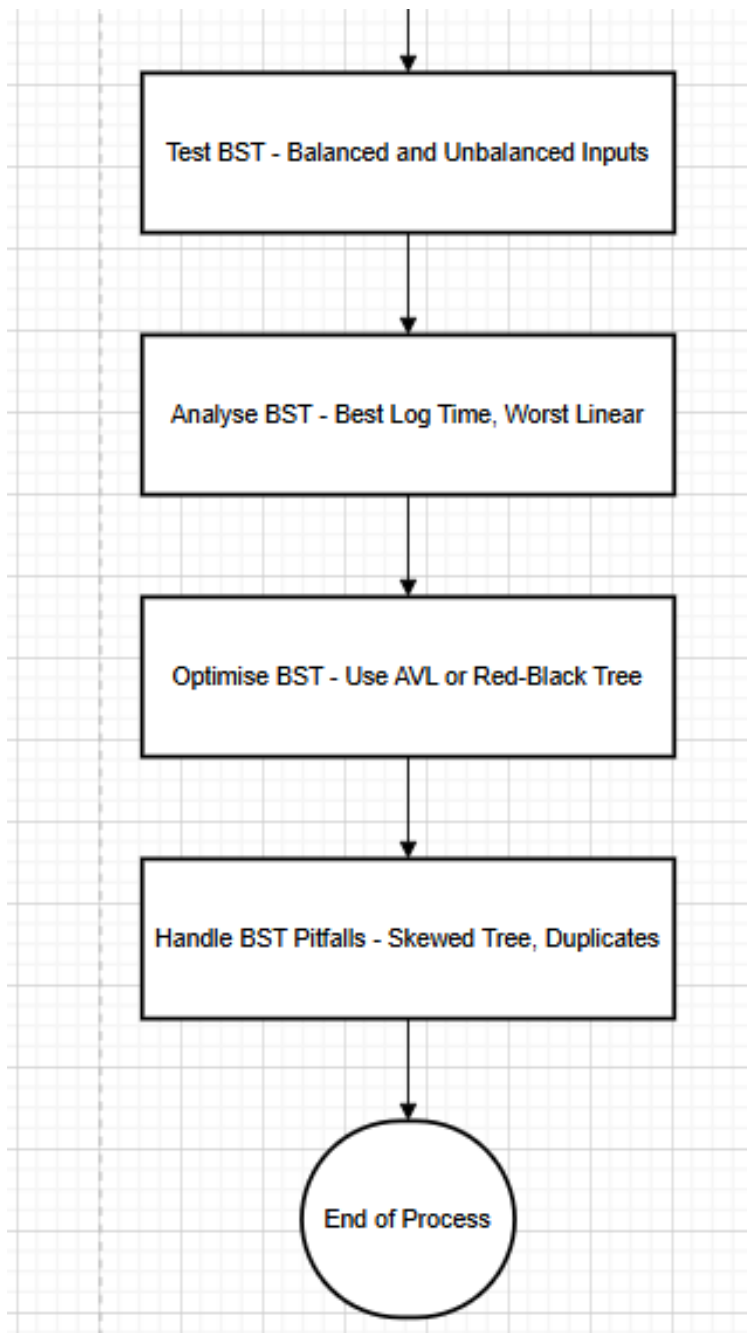
Process Table

It illustrates the process map with the structured steps in programming data structures, implementing abstract data types, and estimating algorithmic complexity. The stages are designed to ensure systematic problem-solving, from initial requirements to final optimization.

Process Map: Stack and Binary Search Tree Construction Workflow







Programming Data Structures

Data structures are essential in programming as they define the way data is stored, accessed, and modified. One of the most efficient and widely used structures is the Binary Search Tree (BST), which offers efficient insertion, searching, and deletion while maintaining a sorted structure (GeeksforGeeks, 2023).

Step 1: Understanding the Problem

According to Goodrich et al.(2013), the objective is to implement a data structure that:

- Maintains sorted order
- Supports logarithmic time complexity for core operations

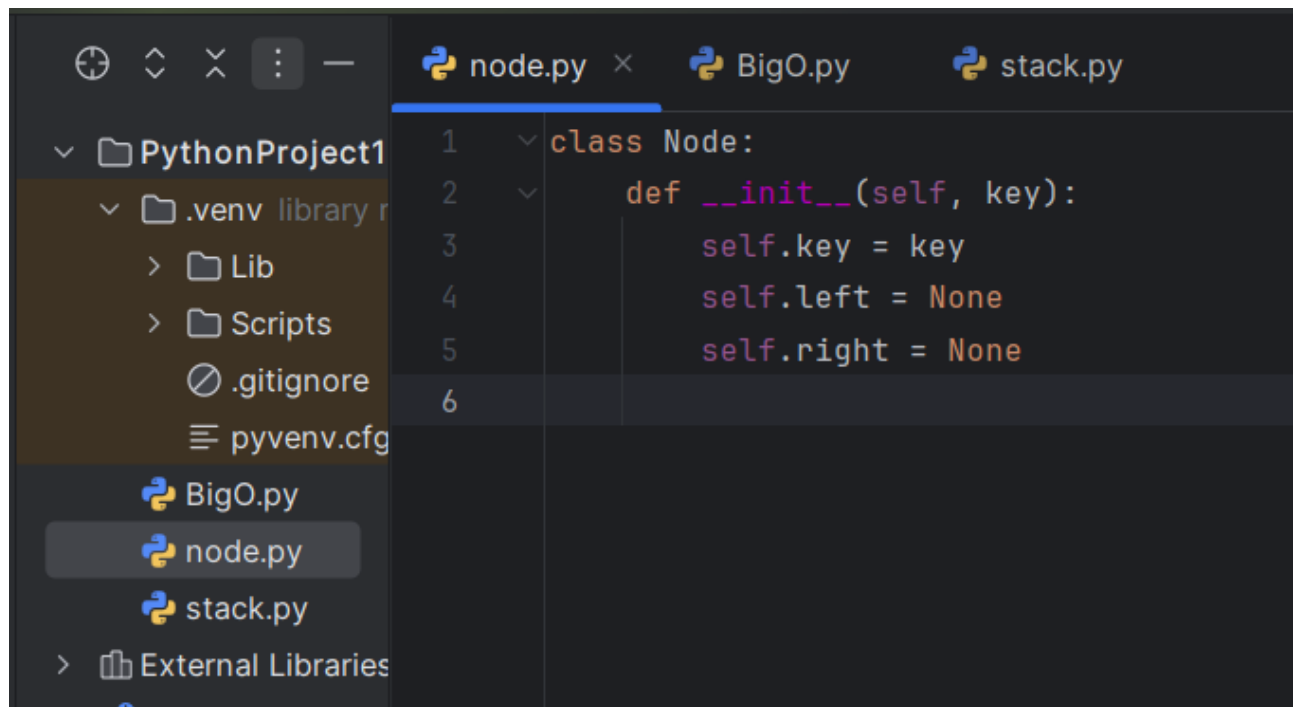
Suitable for:

- Auto-complete systems
- Database indexing

Step 2: Defining the Node Structure

Each node in the BST contains:

- A key (value)
- A left pointer for smaller values
- A right pointer for greater values



The screenshot shows a code editor with a dark theme. On the left is a file explorer for 'PythonProject1' containing a '.venv' directory and files like 'Lib', 'Scripts', '.gitignore', and 'pyvenv.cfg'. Below these are Python files: 'BigO.py', 'node.py' (selected), and 'stack.py'. The main editor area shows the 'node.py' file with the following code:

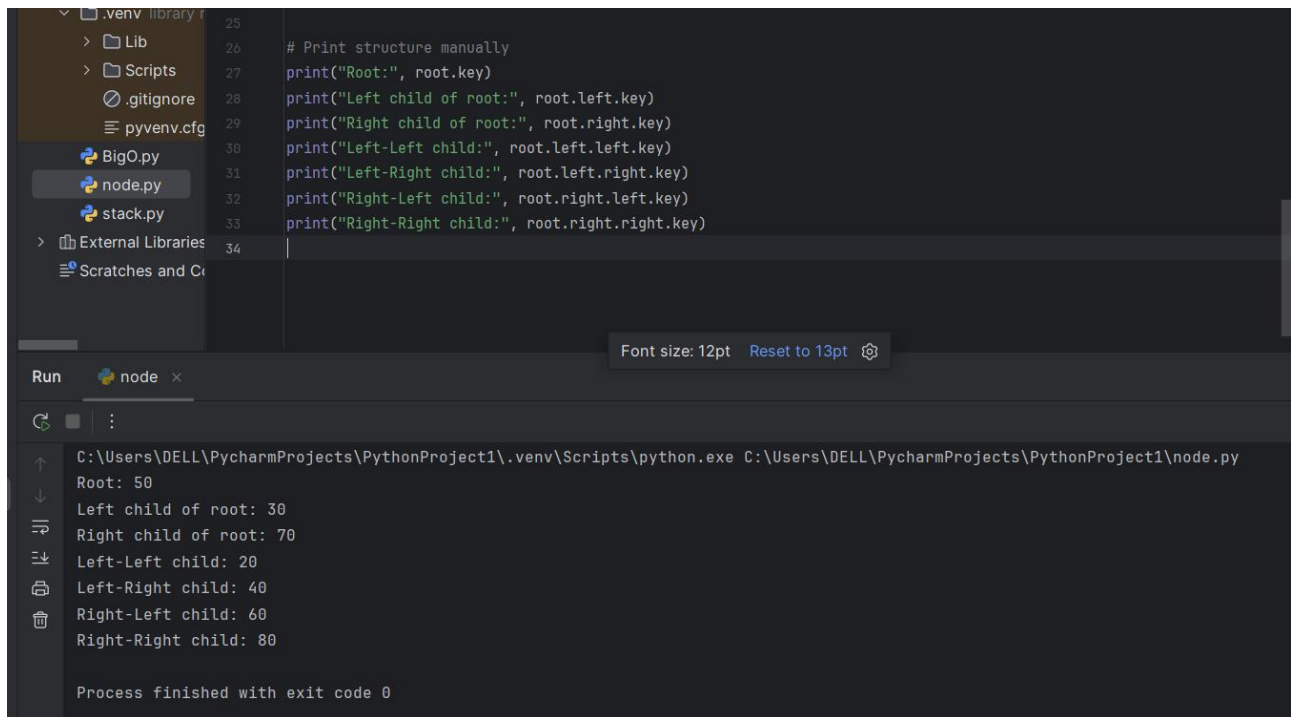
```
1 class Node:
2     def __init__(self, key):
3         self.key = key
4         self.left = None
5         self.right = None
6
```

This creates the foundational building block of the BST.

3. Implementing the Insert Structure

The insert() function recursively places each new value in the correct position according to BST rules.

```
BigO.py  node.py  stack.py
1  # Define the Node class
2  class Node: 1 usage
3      def __init__(self, key):
4          self.key = key
5          self.left = None
6          self.right = None
7
8  # Insert function
9  def insert(root, key): 3 usages
10     if root is None:
11         return Node(key)
12     if key < root.key:
13         root.left = insert(root.left, key)
14     else:
15         root.right = insert(root.right, key)
16     return root
17
18 # ----- Insert and Print Demo -----
19 root = None
20 values = [50, 30, 70, 20, 40, 60, 80]
21
22 # Insert values into BST
23 for val in values:
24     root = insert(root, val)
```

The screenshot shows the PyCharm IDE interface. On the left is the Project Explorer with a file tree containing .venv, Lib, Scripts, .gitignore, pyvenv.cfg, BigO.py, node.py, stack.py, External Libraries, and Scratches and Console. The main editor window displays a Python script with the following code:

```
25  
26 # Print structure manually  
27 print("Root:", root.key)  
28 print("Left child of root:", root.left.key)  
29 print("Right child of root:", root.right.key)  
30 print("Left-Left child:", root.left.left.key)  
31 print("Left-Right child:", root.left.right.key)  
32 print("Right-Left child:", root.right.left.key)  
33 print("Right-Right child:", root.right.right.key)  
34
```

Below the editor is a Run toolbar with a 'Run' button and a 'node' tab. The Run console shows the output of the script:

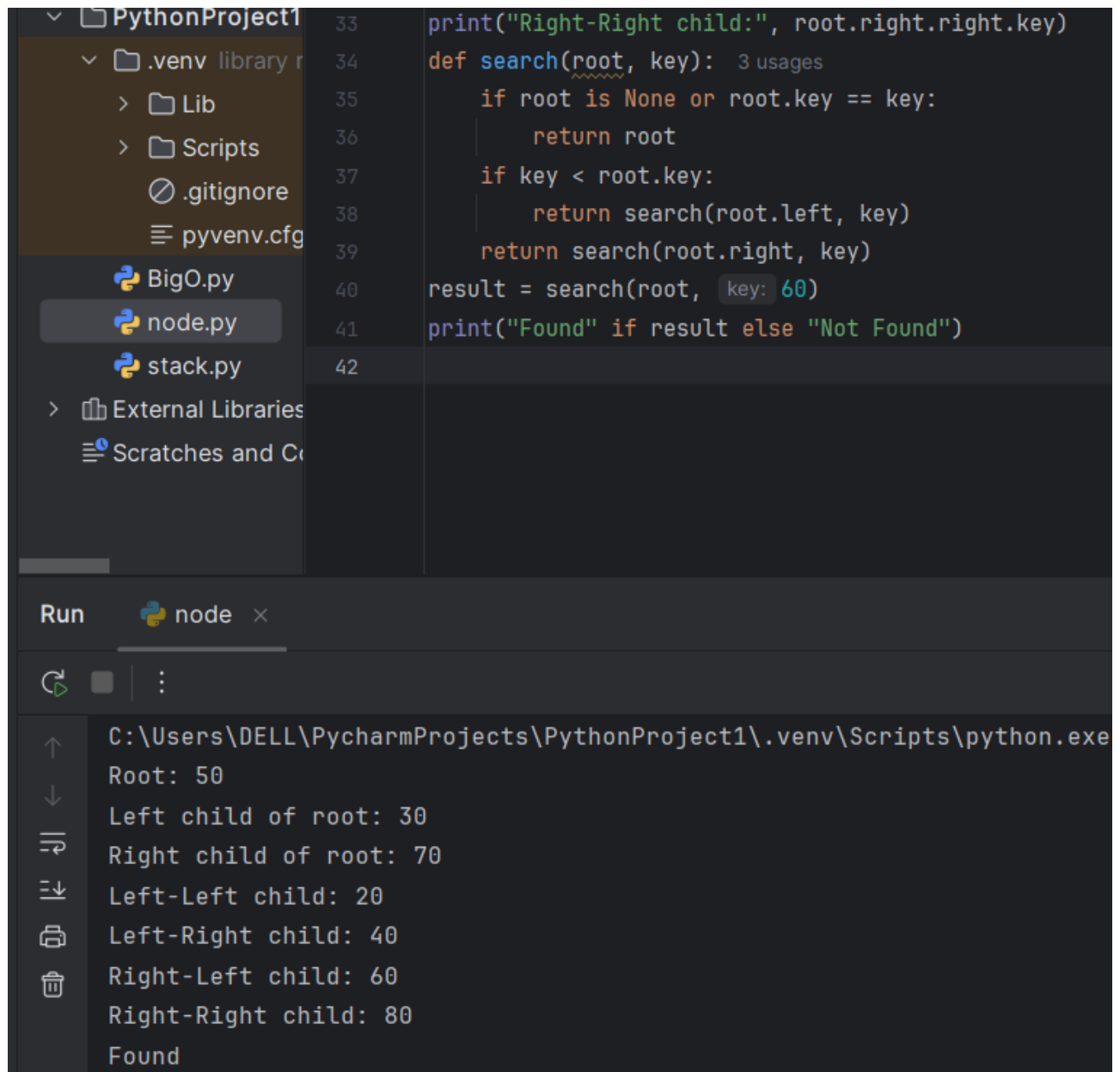
```
C:\Users\DELL\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\DELL\PycharmProjects\PythonProject1\node.py  
Root: 50  
Left child of root: 30  
Right child of root: 70  
Left-Left child: 20  
Left-Right child: 40  
Right-Left child: 60  
Right-Right child: 80  
  
Process finished with exit code 0
```

At the bottom of the Run console, there is a font size indicator showing 'Font size: 12pt' and a 'Reset to 13pt' button.

This ensures that all smaller values go to the left, and larger values go to the right.

4. Implementing the Search in Code

To check if a value exists in the tree:



The screenshot shows the PyCharm IDE with a Python project named 'PythonProject1'. The file explorer on the left shows the project structure, including a '.venv' directory and several Python files. The main editor displays a Python script with the following code:

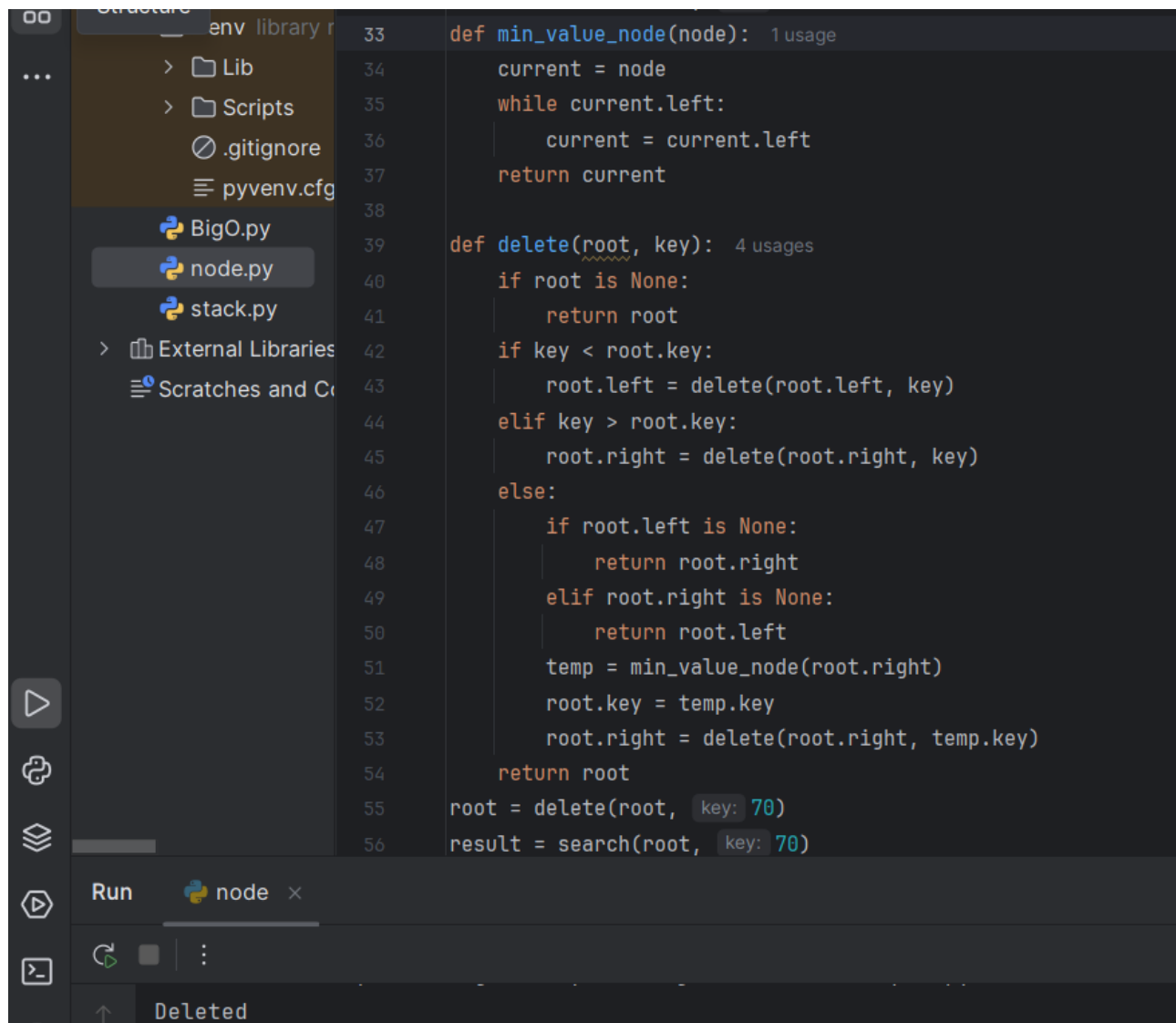
```
33 print("Right-Right child:", root.right.right.key)
34
35 def search(root, key): 3 usages
36     if root is None or root.key == key:
37         return root
38     if key < root.key:
39         return search(root.left, key)
40     return search(root.right, key)
41 result = search(root, key: 60)
42 print("Found" if result else "Not Found")
```

The Run tool window at the bottom shows the output of the program, indicating that the search was successful for the key 60.

```
C:\Users\DELL\PycharmProjects\PythonProject1\.venv\Scripts\python.exe
Root: 50
Left child of root: 30
Right child of root: 70
Left-Left child: 20
Left-Right child: 40
Right-Left child: 60
Right-Right child: 80
Found
```

4. Implementing Deletion

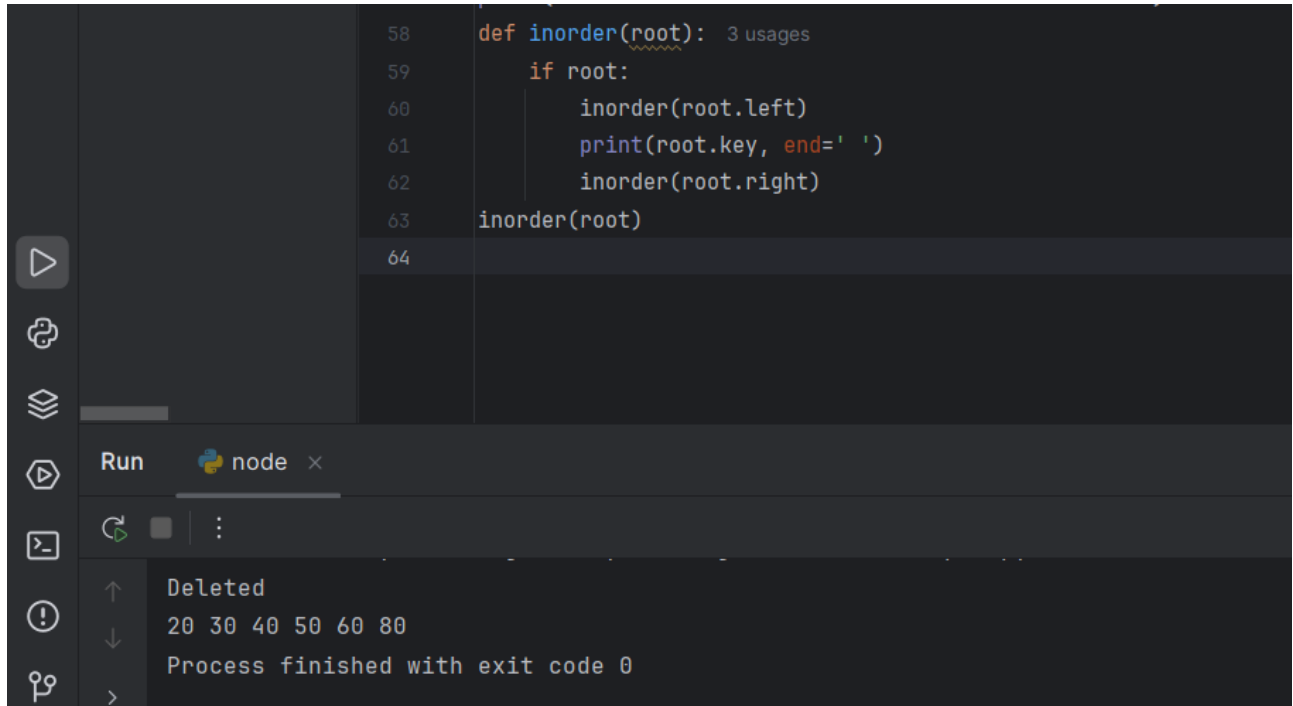
Handle all three cases: leaf, one child, two children.



```
root = delete(root, key: 70)
result = search(root, key: 70)
print("Deleted" if result is None else "Still Exists")
```

Step 5: Implement In-Order Traversal (to Display BST)

Displays the tree's contents in sorted order.



The screenshot shows a code editor with a dark theme. The code defines an `inorder` function that recursively traverses a binary search tree. The function calls `inorder(root.left)`, prints the root's key, and then calls `inorder(root.right)`. The output console shows the sorted keys: 20, 30, 40, 50, 60, 80. The number 70 is missing from the output, indicating it was deleted earlier. The console also shows 'Deleted' and 'Process finished with exit code 0'.

```
58 def inorder(root): 3 usages
59     if root:
60         inorder(root.left)
61         print(root.key, end=' ')
62         inorder(root.right)
63 inorder(root)
64
```

Run node x

Deleted
20 30 40 50 60 80
Process finished with exit code 0

Here 70 is missing as I have deleted it earlier.

Step 7: Optimize the Tree Structure

- Use AVL Trees or Red-Black Trees to ensure the tree stays balanced
- Auto-balancing keeps complexity close to $O(\log n)$ even in worst-case insertions

Step 8: Handle Pitfalls and Finalize

Pitfall	Handling Strategy
Skewed Tree	Use self-balancing (AVL)
Incorrect Deletion	Use in-order successor properly
Duplicate Keys	Define behaviors
Null References	Add safety checks before accessing children

Implementing Abstract Data Types (ADTs)

Implementing ADTs improves modularity, reusability, and maintainability in software systems (GeeksforGeeks 2023). Here we will use Stack Data Type.

Step 1: Understand the Problem and Choose LIFO Logic

A stack is needed when:

- The last item added should be the first removed.
- Common use cases: undo feature, browser history.

Goal: Implement a Last-In-First-Out (LIFO) structure.

Step 2: Defining the ADT Interface

Four keys operations supporting stack:

- `push(item)` – add item to the top
- `pop()` – remove item from the top
- `peek()` – view top item
- `is_empty()`: Check whether the stack is empty

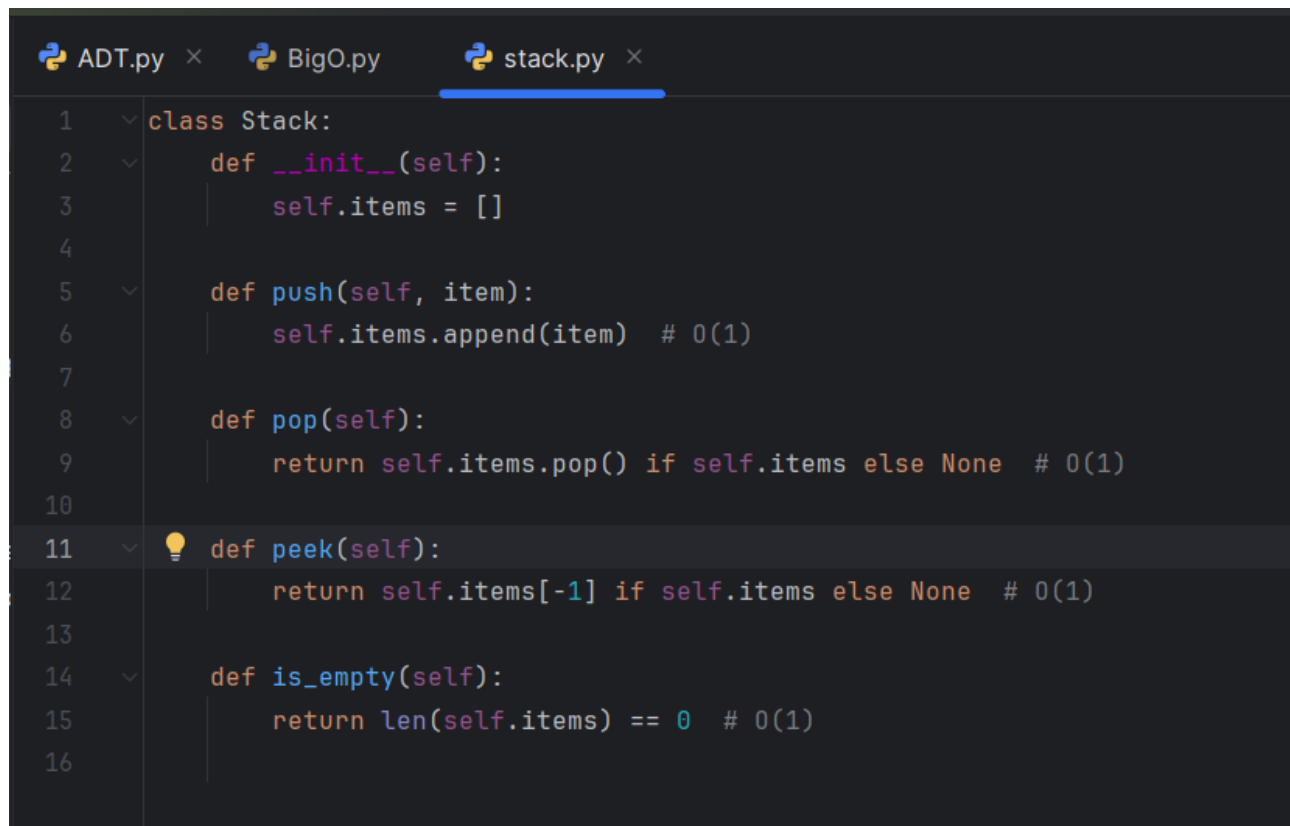
Step 3: Choosing the Underlying Data Structure

Depending on performance needs, a Stack can be implemented:

- Efficient `append()` for push
- Efficient `pop()` from the end

Step 4. Implementing the Operations

Here's a simple Python class that shows how a Stack can be implemented as an ADT:



```
1  class Stack:
2      def __init__(self):
3          self.items = []
4
5      def push(self, item):
6          self.items.append(item) # O(1)
7
8      def pop(self):
9          return self.items.pop() if self.items else None # O(1)
10
11     def peek(self):
12         return self.items[-1] if self.items else None # O(1)
13
14     def is_empty(self):
15         return len(self.items) == 0 # O(1)
16
```

Step 5. Test and Implementation

Testing the above code in Py-charm with:

- Pushing multiple elements
- Checking top with peek()
- Removing elements with pop()
- Checking for empty stack

The screenshot shows a PyCharm IDE with a project named 'PythonProject1'. The file explorer on the left shows files: .gitignore, pyenvv.cfg, ADT.py, BigO.py, stack.py, External Libraries, and Scratches and Console. The main editor displays the code for 'stack.py':

```
19 s = Stack()
20 print(s.is_empty())
21 s.push(10)
22 s.push(20)
23 print(s.peak())
24 print(s.pop())
25
```

Below the editor is the 'Run' window, which shows the execution of 'stack.py'. The output is:

```
C:\Users\DELL\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\DELL\PycharmProjects\PythonProject1\stack.py
True
20
20
10
True
Process finished with exit code 0
```

Step 6: Handle Pitfalls and Finalize

This final step ensures the data structure is error-tolerant, stable, and ready for integration into larger systems as it is critical to identify potential pitfalls and apply defensive programming techniques to ensure robustness.

Pitfall	Description
Underflow	Occurs when trying to pop() or peek() from an empty stack
Incorrect Top Reference	Trying to access the top item using items[0] instead of items[-1]
Lack of Input Validation	Not checking whether the input is valid

```
1 class StackADT:
2     def __init__(self):
3         # Internal list used to store items (not directly accessible)
4         self._items = []
5
6     def push(self, item):
7         """Add item to the top of the stack"""
8         self._items.append(item)
9
10    def pop(self):
11        """Remove and return the top item"""
12        if self.is_empty():
13            return "Stack is empty"
14        return self._items.pop()
15
16    def peek(self):
17        """Return the top item without removing it"""
18        if self.is_empty():
19            return "Stack is empty"
20        return self._items[-1]
21
22    def is_empty(self): 2 usages
23        """Check if the stack is empty"""
24        return len(self._items) == 0
25
26    def size(self):
27        """Return the number of items in the stack"""
28        return len(self._items)
29
30
```

```
29
30 stack = StackADT()
31 stack.push(100)
32 stack.push(200)
33 print("Top item:", stack.peek())
34 print("Removed:", stack.pop())
35 print("Size:", stack.size())
36
```



```
C:\Users\DELL\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\DELL\PycharmProjects\PythonProject1\ADT.py
Top item: 200
Removed: 200
Size: 1

Process finished with exit code 0
```

Estimating Algorithmic Complexity

After integrating the data structure or ADT into the system, the next essential step is to evaluate how efficiently it performs. When designing efficient software, it's not enough to just make a program work it must also perform well under different conditions. According to Cormen et al. (2009) Algorithmic complexity is a measure of how an algorithm's resource usage (like execution time or memory) grows with input size.

Binary Search Tree (BST)

The Binary Search Tree is a hierarchical data structure that maintains sorted elements for fast access. Its efficiency depends on whether it's balanced or skewed.

Operation	Best Case	Average Case	Worst Case	Explanation
insert()	$O(\log n)$	$O(\log n)$	$O(n)$	Depth of the tree determines cost
search()	$O(\log n)$	$O(\log n)$	$O(n)$	Faster in balanced trees
delete()	$O(\log n)$	$O(\log n)$	$O(n)$	Involves search + re-balancing
inorder()	$O(n)$	$O(n)$	$O(n)$	Visits every node exactly once

The worst case occurs when the tree becomes unbalanced, such as inserting sorted values into an empty BST (forming a linked list).

Stack (Abstract Data Type)

The Stack ADT uses a Last-In-First-Out (LIFO) approach and is typically implemented using a list or a linked list. Its operations are highly efficient because they work only at one end of the structure (Sommerville 2016).

OPERATION	TIME COMPLEXITY	EXPLANATION
PUSH()	O(1)	Appends item to top of stack
POP()	O(1)	Removes top item instantly
PEEK()	O(1)	Accesses top without removing
IS_EMPTY()	O(1)	Check if the stack is empty

All core operations run in constant time, making the stack ideal for scenarios where speed and simplicity are critical.

Both the Binary Search Tree (BST) and Stack ADT demonstrate how different data structures can impact the efficiency of a program.

- Stack is highly efficient with constant time complexity ($O(1)$) for all core operations, making it ideal for tasks that require fast, sequential access like undo functionality or parsing expressions.
- BST provides fast access to sorted data with logarithmic complexity ($O(\log n)$) in the best and average cases. However, its performance can degrade to linear time ($O(n)$) if the tree becomes unbalanced.

Understanding these complexities ensures developers make informed choices when selecting data structures based on performance needs, input size, and usage patterns(Big-O Cheat Sheet 2024).

Conclusion

This report demonstrated the construction and implementation of a Stack and a Binary Search Tree (BST). The Stack, with its simple LIFO structure, offers constant time performance for all core operations. In contrast, BST provides efficient data management through sorted insertion and retrieval, especially when balanced. Understanding their design, performance, and use cases is essential for writing efficient and reliable programs.

References

Cormen, TH, Leiserson, CE, Rivest, RL & Stein, C 2009, *Introduction to algorithms*, 3rd edn, The MIT Press, Cambridge, MA.

Goodrich, MT, Tamassia, R & Goldwasser, MH 2014, *Data structures and algorithms in Python*, 1st edn, Wiley, Hoboken, NJ

GeeksforGeeks 2023, *Abstract data types (ADT)*, GeeksforGeeks, viewed 15 May 2025, <https://www.geeksforgeeks.org/abstract-data-types/>.

GeeksforGeeks 2023, *Binary Search Tree (BST)*, GeeksforGeeks, viewed May 23, 2025, <https://www.geeksforgeeks.org/binary-search-tree-data-structure/>

Big-O Cheat Sheet 2024, *Time complexity of algorithms*, Big-O Cheat Sheet, viewed 15 May 2025, <https://www.bigocheatsheet.com/>.

Shaffer, CA 2013, *Data structures and algorithm analysis in Java*, 3rd edn, Dover Publications, Mineola, NY.

Sommerville, I 2016, *Software engineering*, 10th edn, Pearson, Harlow.